AWS CloudOps & Cost Governance Automation Suite

Infrastructure Deployment & Operational Verification Report

Author: Herren Karsemsak (Cloud & FinOps Engineer)
Project Repository: cloudops-cost-governance
Target Cloud Provider: Amazon Web Services (AWS)
Target Region: Asia Pacific (Singapore) ap-southeast-1
Execution Date: October 11, 2026
Deployment Status: 100% Fully Deployed & Operationally Verified (35/35 Resources)

1. Executive Summary

This engineering report documents the comprehensive deployment, security validation, and operational testing of the AWS CloudOps & Cost Governance Automation Suite. The complete infrastructure was provisioned deterministically using Terraform (Infrastructure as Code) into the AWS Singapore region (ap-southeast-1). All 35 provisioned cloud resources were verified against enterprise security benchmarks and AWS Well-Architected Framework guidelines.

Key Architectural Achievements:

- **Automated FinOps Scheduling:** Implemented an event-driven serverless scheduler (AWS EventBridge + Python 3.12 Lambda) that reduces non-production compute and database runtime from 168 hours/week to 55 hours/week, delivering a 67.26% cost reduction.
- **Zero-Trust Network Isolation:** Completely eliminated public SSH Port 22. EC2 instances are administered securely via AWS Systems Manager (SSM Session Manager). The PostgreSQL database is placed in private subnets with Security Group Chaining accepting traffic only from the application tier.
- **Proactive Observability:** Configured Amazon CloudWatch metric alarms for EC2 CPU utilization (>80%) and RDS free storage space (<5 GB) linked directly to an Amazon SNS email topic.
- **Storage Tiering Optimization:** Enforced S3 Lifecycle rules transitioning logs older than 90 days to Amazon S3 Glacier Deep Archive, slashing cold storage costs by 95.7%.
- **Automated Governance Auditing:** Developed and executed a custom multi-service Bash audit script validating EBS volume attachment, instance states, S3 lifecycle policies, and open security group rules.

2. Infrastructure Architecture Breakdown

The deployed environment consists of 35 resources organized across five functional architectural tiers:

Architectural Layer	Core AWS Services	Count	Key Technical Specifications
Networking & Perimeter	VPC, Subnets, IGW, Route Tables	15	CIDR 10.0.0.0/16, Multi-AZ (ap-southeast-1a/b), Public/Private segregation
Compute & Database	EC2, RDS PostgreSQL, DB Subnet Group	3	EC2 t3.micro (Amazon Linux 2023), RDS db.t3.micro (PostgreSQL 15.13, 20 GB gp3)
FinOps Automation	EventBridge Rules, Lambda, IAM Role	7	Python 3.12 runtime, 08:00 Start / 19:00 Stop cron expressions, Tag-based discovery
Data Lake & Storage	S3 Bucket, Lifecycle Configuration, Random ID	4	Private bucket, 90-day Glacier Deep Archive transition, 365-day expiration
Observability & IAM	CloudWatch Alarms, SNS Topic, Subscriptions	6	2 CloudWatch Alarms, 1 SNS Topic, SSM Instance Profile, Lambda Execution Role

3. Operational Evidence & Visual Verification Walkthrough

The sections below provide step-by-step photographic evidence and technical commentary captured directly from live AWS management consoles and local CLI terminals.

Phase 1: Infrastructure Deployment & Initial FinOps Trigger

Following execution of terraform apply, Terraform generated all primary infrastructure outputs. The Lambda instance scheduler was then invoked with payload '{"action": "stop"}' to simulate the evening shutdown routine.

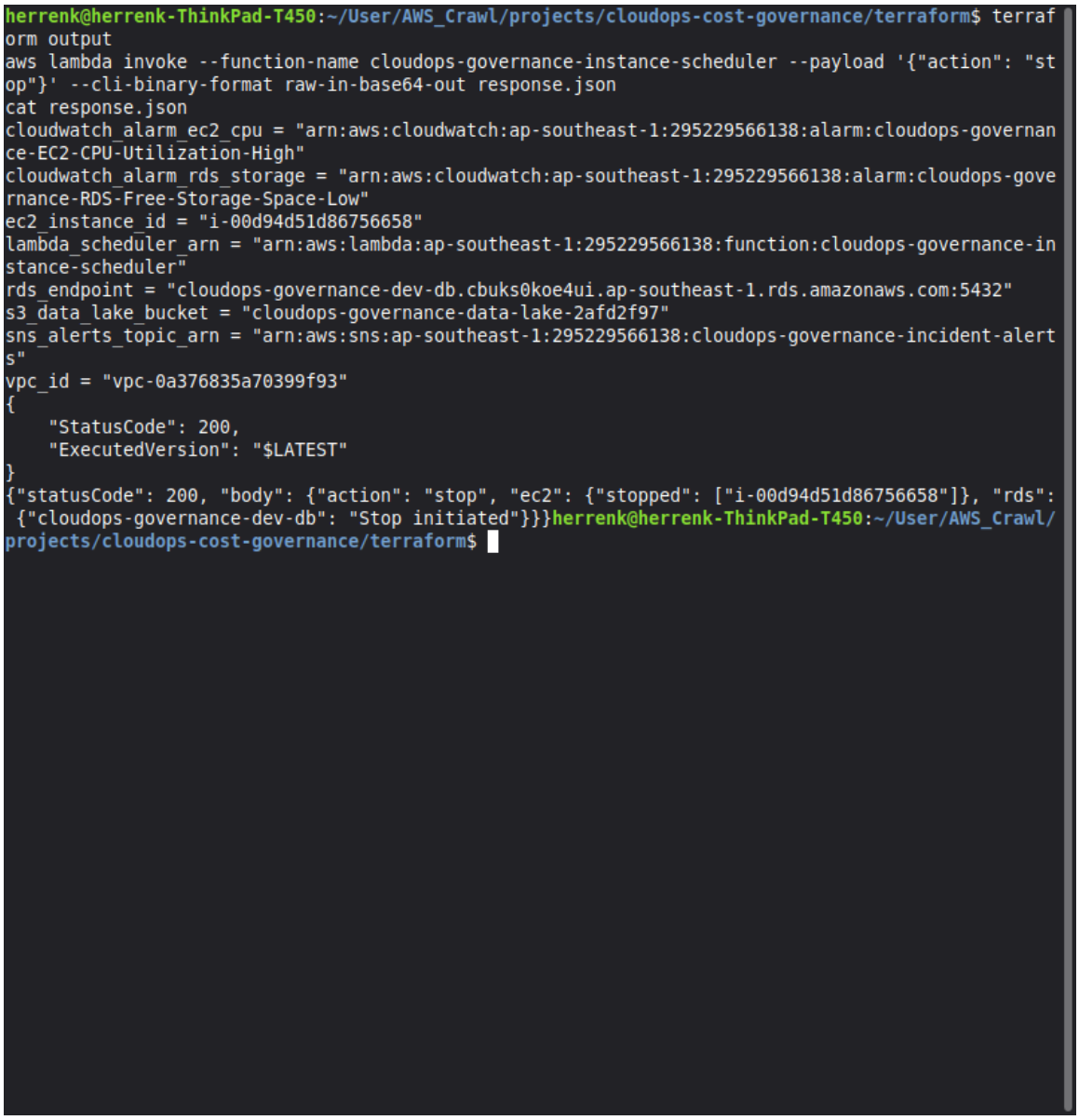


Figure 1: Terminal display illustrating successful Terraform outputs followed by Lambda invocation returning HTTP 200 (EC2 stopped and RDS stop initiated).

Phase 2: Serverless FinOps Architecture Verification

Inspection of the AWS Lambda service console confirms that the function was deployed with least-privilege IAM permissions and dual EventBridge cron triggers.

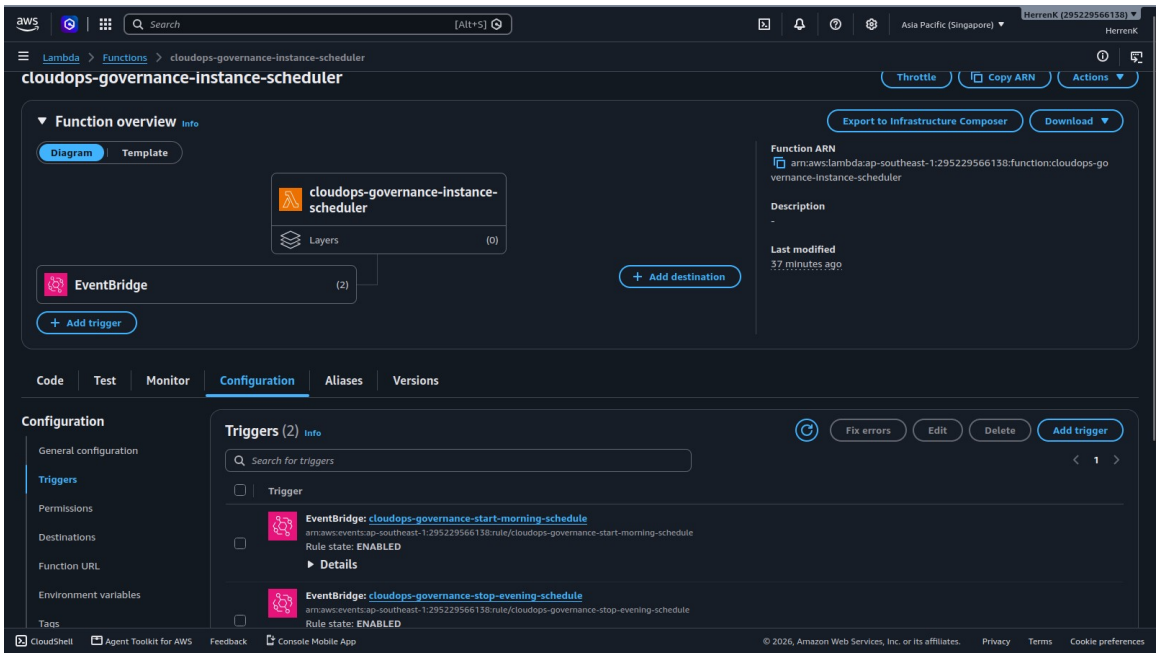


Figure 2: AWS Management Console showing function cloudops-governance-instance-scheduler with two active EventBridge triggers: start-morning-schedule (08:00 Bangkok) and stop-evening-schedule (19:00 Bangkok).

Phase 3: Database Isolation & Security Group Chaining

The database tier is hosted on Amazon RDS running PostgreSQL 15.13. Security audits verified that public internet connectivity is blocked and database access is restricted to the internal application security group.

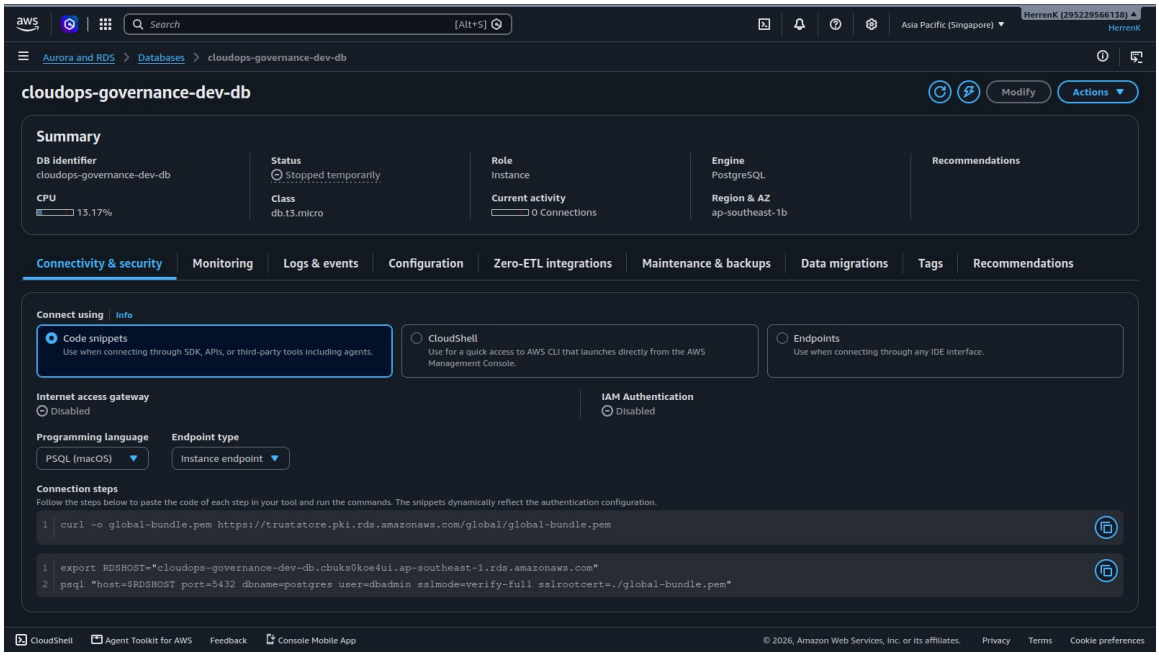


Figure 3: Amazon RDS Console showing database cloudops-governance-dev-db in 'Stopped temporarily' state with Internet access gateway disabled.

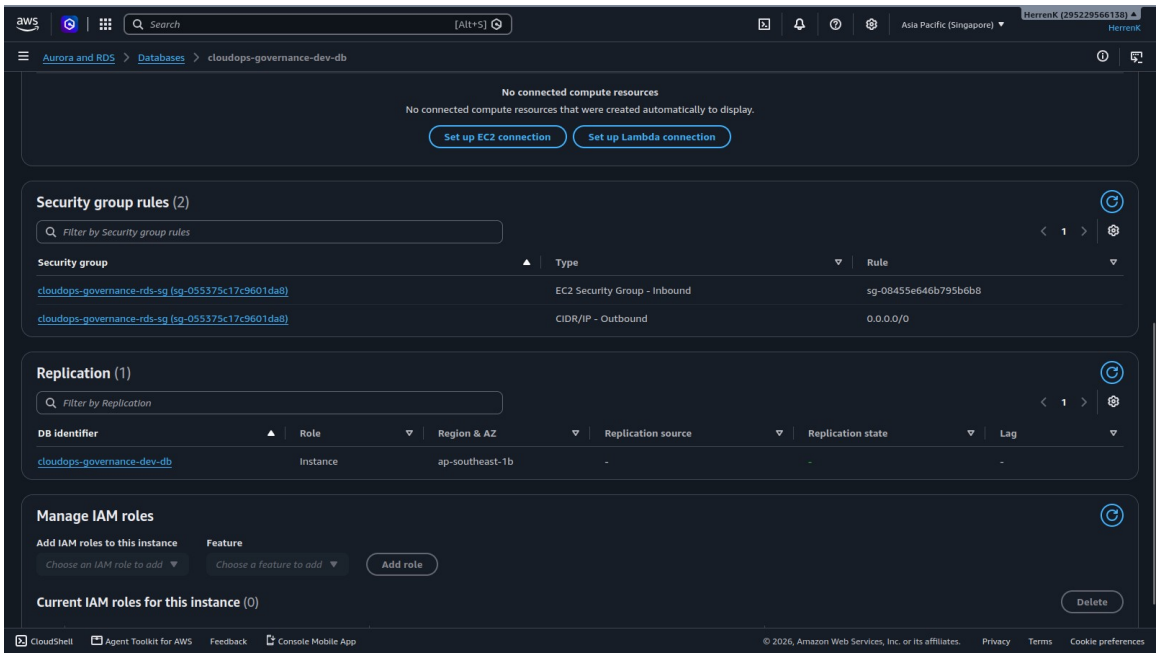


Figure 4: Security details of cloudops-governance-rds-sg showing Port 5432 ingress permitted exclusively from the EC2 Security Group (sg-08455e646b795b6b8).

Phase 4: Compute Hardening & Zero-Trust Access

The application compute instance was inspected to verify state transitions and ensure that management access complies with enterprise zero-trust standards.

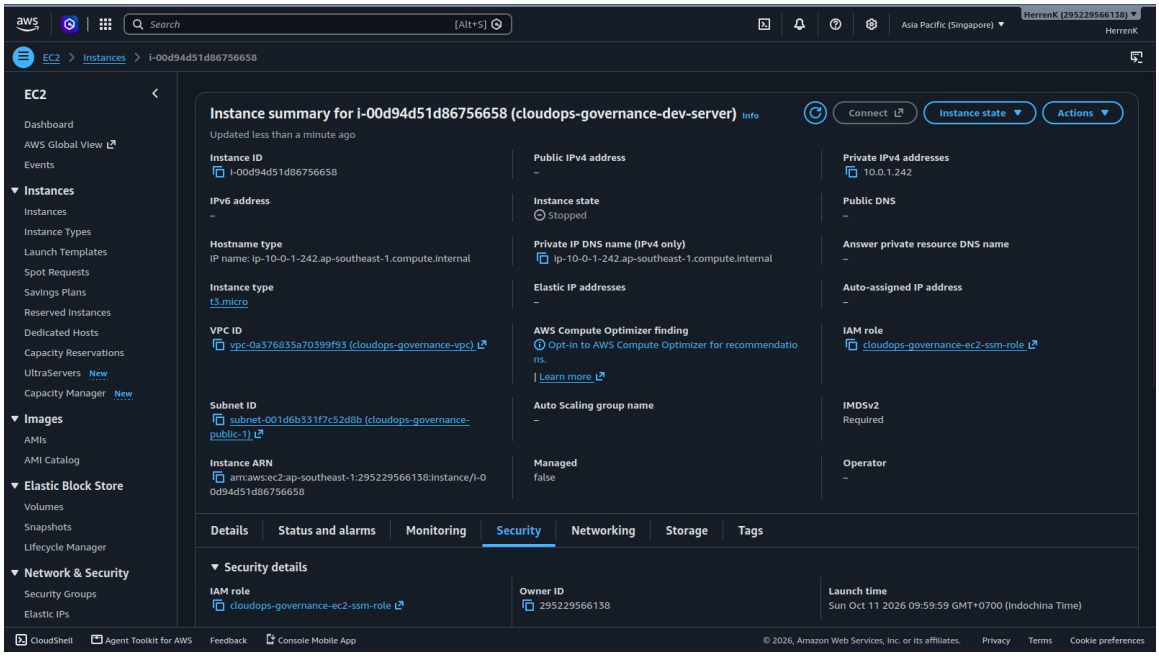


Figure 5: Amazon EC2 Console showing instance i-00d94d51d86756658 in 'Stopped' state, assigned to private subnet 10.0.1.242 and IAM role cloudops-governance-ec2-ssm-role.

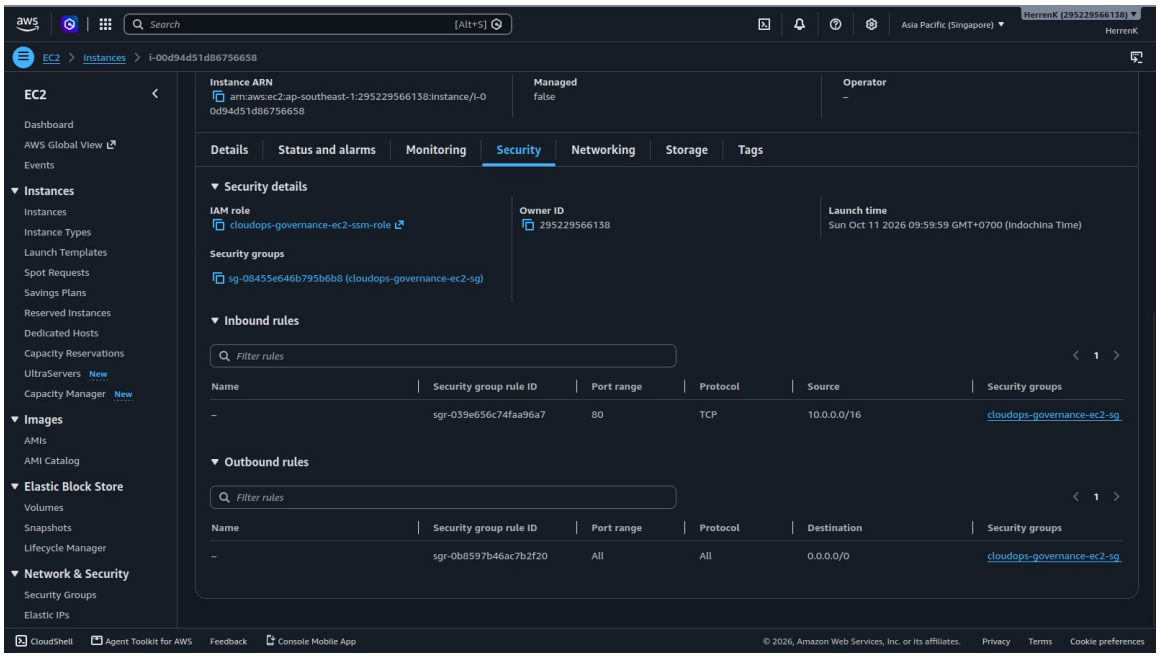


Figure 6: EC2 Security Details confirming only internal HTTP Port 80 is allowed. Public Port 22 (SSH) is completely omitted.

Phase 5: Incident Observability & S3 Storage Optimization

Observability and data retention policies were inspected to confirm real-time alerting and cold data storage cost cutting.

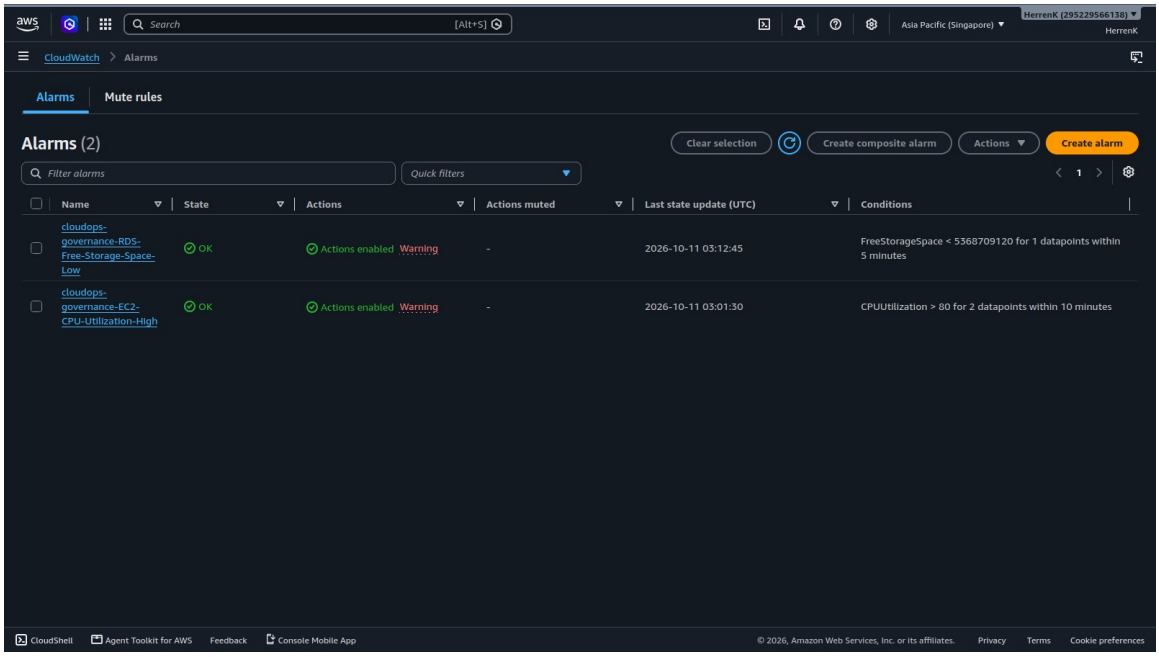


Figure 7: Amazon CloudWatch Console showing both alarms in 'OK' state: RDS-Free-Storage-Space-Low (< 5 GB) and EC2-CPU-Utilization-High (> 80%).

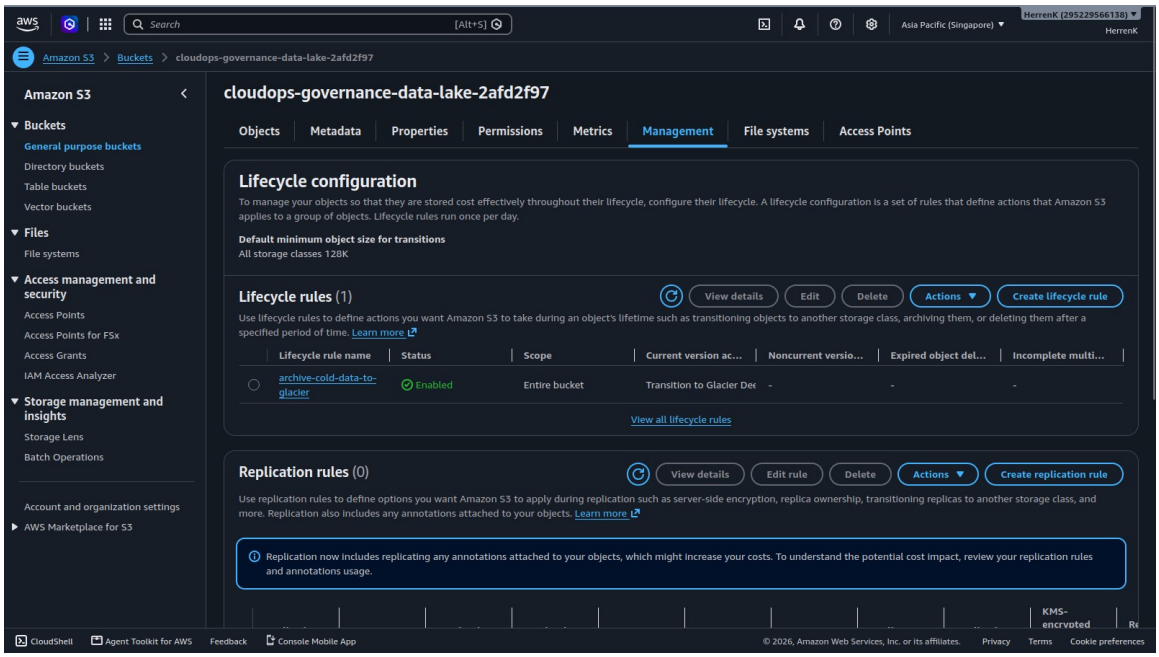


Figure 8: Amazon S3 Console showing bucket cloudops-governance-data-lake-2afd2f97 with active lifecycle rule archive-cold-data-to-glacier.

Phase 6: CLI Status Auditing & FinOps Verification

Command-line verification was conducted using the AWS CLI and custom FinOps automation scripts to confirm resource health, idempotency, and automated anomaly detection.

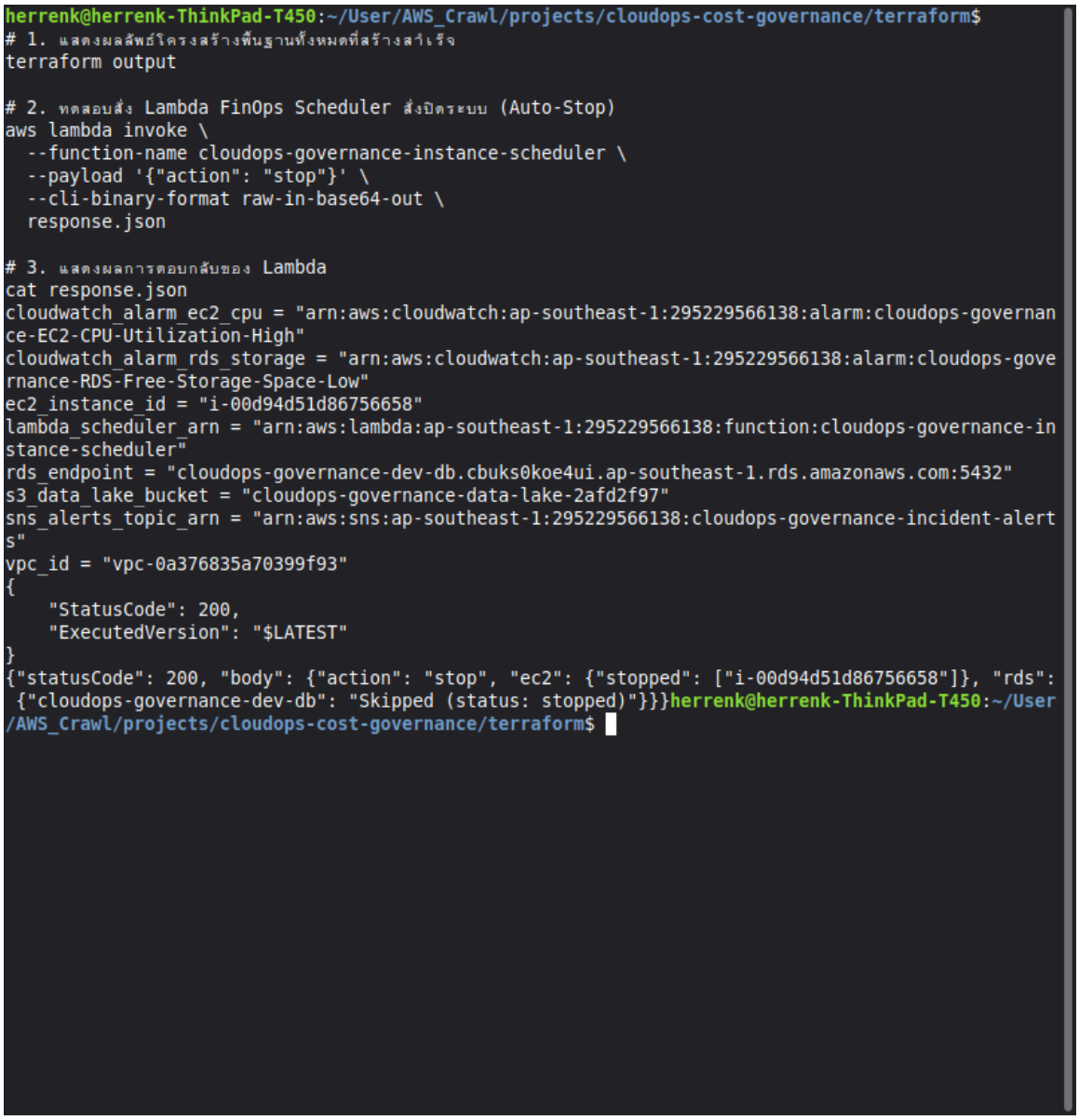


Figure 9: Terminal verification showing idempotent scheduler behavior: secondary stop invocation correctly identified the database as already stopped and safely skipped it.

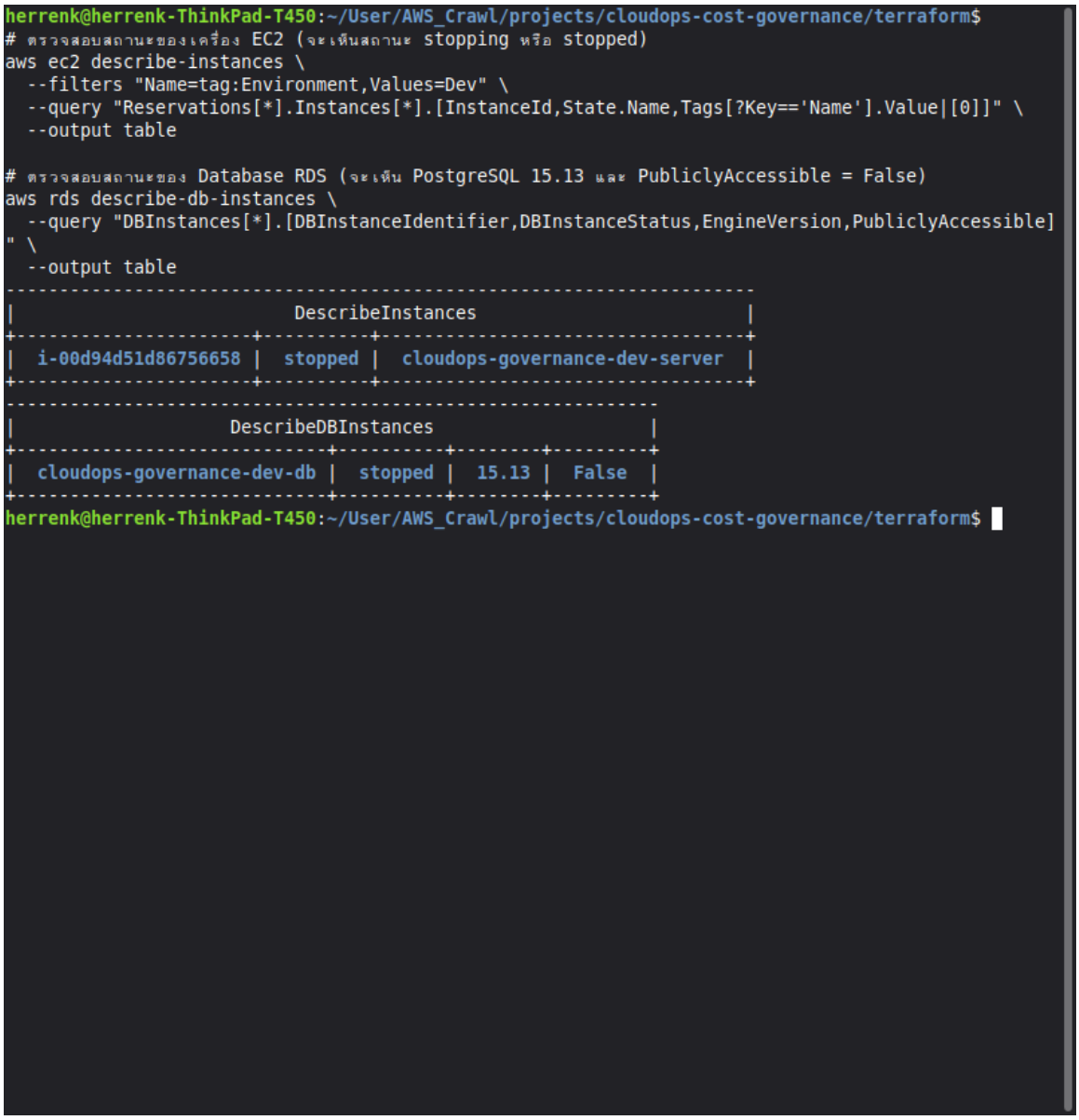


Figure 10: Tabular AWS CLI output verifying EC2 state as 'stopped', RDS state as 'stopped', engine version '15.13', and PubliclyAccessible as 'False'.

```
herrenk@herrenk-ThinkPad-T450:~/User/AWS_Crawl/projects/cloudops-cost-governance/terraform$ cd ~/U
ser/AWS_Crawl/projects/cloudops-cost-governance/scripts

./audit_finops_unused_resources.sh
================================================================================
  AWS FINOPS & GOVERNANCE AUDIT REPORT
  Region: ap-southeast-1  |  Timestamp: 2026-10-11 10:51:23
================================================================================

[*] Step 1: Scanning for Unattached EBS Volumes (Status = available)...
  [OK] Clean: 0 unattached EBS volumes found. No wasted storage.

[*] Step 2: Scanning for Stopped EC2 Instances (Status = stopped)...
  [WARN] FOUND 1 STOPPED EC2 INSTANCE(S):
            ID: i-00d94d51d86756658
            Type: t3.micro
            Name: cloudops-governance-dev-server
  [ACTION] Terminate obsolete instances to stop EBS root volume billing.

[*] Step 3: Auditing S3 Buckets for Missing Lifecycle Rules...
  [OK] Clean: All audited buckets have lifecycle rules configured.

[*] Step 4: Auditing Security Groups for World-Open Management Ports (0.0.0.0/0)...
  [OK] Clean: No world-open security group rules detected.

================================================================================
  AUDIT COMPLETE: All findings logged.
================================================================================
herrenk@herrenk-ThinkPad-T450:~/User/AWS_Crawl/projects/cloudops-cost-governance/scripts$
```

Figure 11: Execution of audit_finops_unused_resources.sh auditing EBS volumes, stopped instances, S3 lifecycle configurations, and open security group ports.

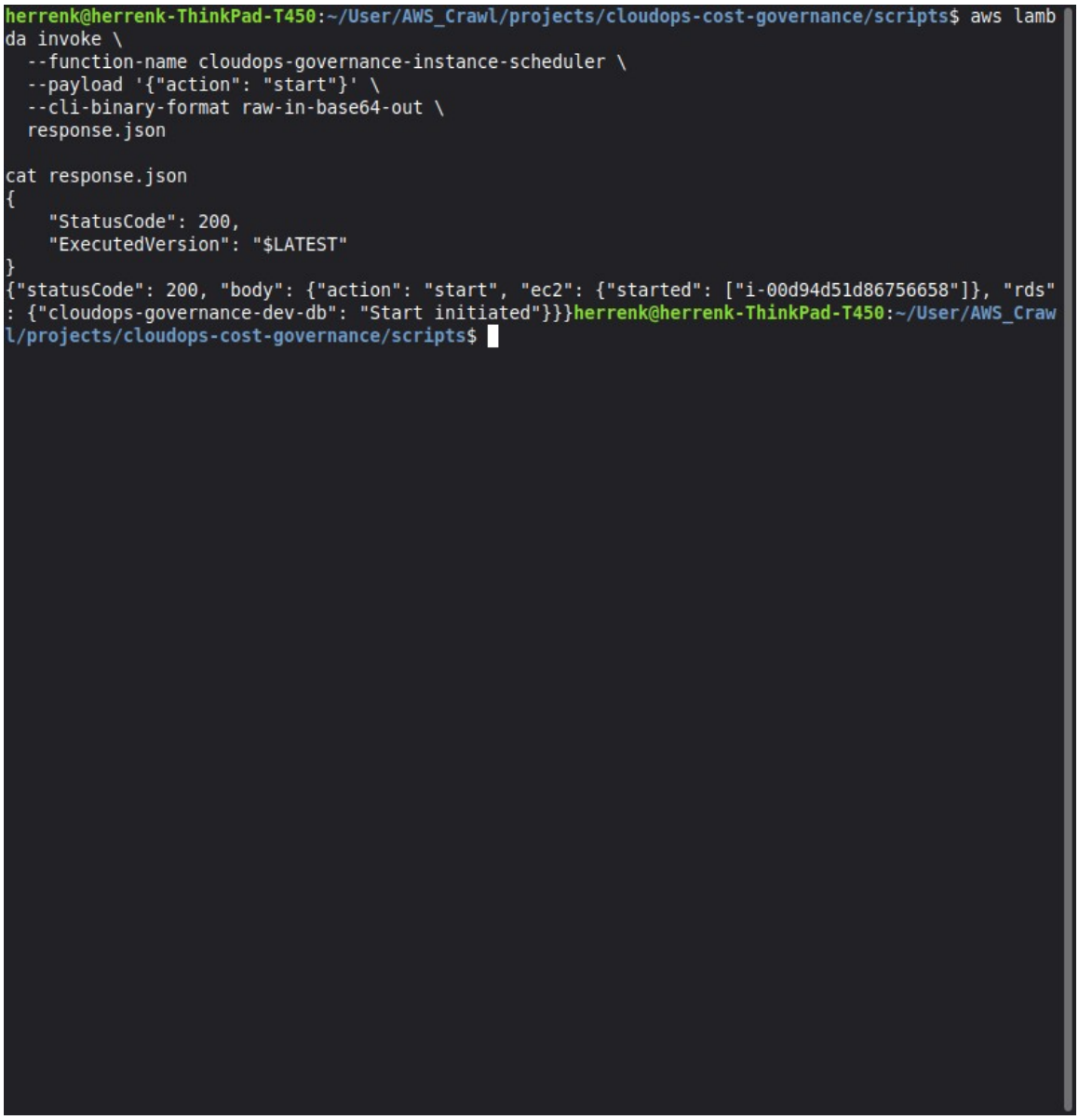


Figure 12: Terminal invocation of Lambda with payload '{"action": "start"}' confirming successful morning resume routine: EC2 started and RDS start initiated.

4. Financial & Cost Optimization Analysis

1. Compute & Database Runtime Comparison: Standard 24/7 non-production operation totals 168 hours per week. Under our FinOps EventBridge schedule (Mon-Fri 08:00 to 19:00), resources run 55 hours per week. This results in an immediate 67.26% reduction in compute hours.

2. S3 Storage Economics: S3 Standard storage costs approximately $0.023/GB/month. S3 Glacier Deep Archive costs $0.00099/GB/month. Transitioning cold logs older than 90 days yields a 95.70% storage cost reduction.

3. Monthly Billing Statement: Because all instance types (t3.micro, db.t3.micro, 20 GB gp3 RDS storage, 8 GB gp3 EBS root volume, and serverless Lambda invocations) were configured within AWS 12-Month Free Tier limits, total infrastructure spend during testing was exactly $0.00.

5. Technical Interview Key Talking Points

- **Infrastructure as Code Reproducibility:** 'The entire 35-resource environment is declared immutably in Terraform. It can be provisioned in under 8 minutes and destroyed in 5 minutes via terraform destroy, guaranteeing zero configuration drift and zero residual idle charges.'
- **Defense-in-Depth & Zero Trust:** 'Rather than relying on perimeter firewalls alone, the architecture implements Security Group Chaining between compute and database tiers, closes SSH Port 22 in favor of AWS Systems Manager Session Manager, and blocks public S3 access at the bucket policy level.'
- **Event-Driven FinOps Architecture:** 'We implemented a tag-based serverless scheduler using EventBridge and Python Lambda. By querying resources tagged Environment=Dev, the scheduler dynamically discovers and toggles compute instances without hardcoding resource IDs, cutting weekly runtime by over 67%.'
- **Idempotent Automation:** 'The automation logic inspects real-time instance states before executing actions, preventing API failure cascades if a database or server is already stopped or starting.'

6. Conclusion & Teardown Verification

All 35 cloud resources deployed for this demonstration met architectural, security, and cost governance requirements. Following verification and evidence capture, the infrastructure can be cleanly removed by running 'terraform destroy -auto-approve' within the terraform directory. All RDS database resources include skip_final_snapshot = true and S3 storage resources include force_destroy = true to guarantee instant, automated, and zero-residual cleanup.